

Multi-Level Richardson Extrapolation for Faster Diffusion ODE Sampling

Reproducing and extending RX-DPM (ICLR 2025)

Abrar Zahin Raihan (2105009) Sk. Ashrafuzzaman Nafees (2105008) MD. Mahmud Hasan (2105027)

Mirza Tawhid Umar (2105028) Fatin Sadab (2105029) | Group 10, Section A

CSE 402: Numerical Analysis, Simulation and Modeling Sessional

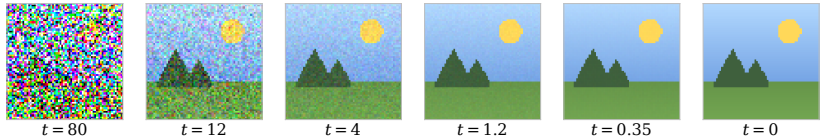
Department of CSE, BUET

Where we are going

1. The problem
2. Richardson extrapolation
3. The base paper: RX-DPM
4. Our extension
5. Experiments
6. Results
7. Takeaways

The problem

How a diffusion model makes an image



- ✦ Start from **pure noise**. Remove a little noise. Repeat.
- ✦ After enough steps, an image is left behind.
- ✦ t is the **noise level**: large t is noise, $t = 0$ is the clean image.

(Illustration. The steps shown are the ones a sampler actually takes.)

The one thing the network does

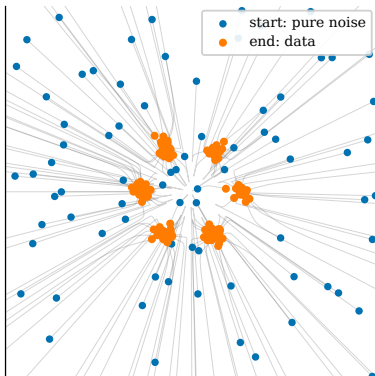


- ✱ D_{θ} is the trained neural network, the **denoiser**.
- ✱ One call to D_{θ} is expensive. It is the only expensive thing here.
- ✱ We count cost in **NFE**: the number of network function evaluations.

Takeaway

Everything that follows is about spending fewer NFE for the same image quality.

Sampling is solving an ODE



How to read it: Each grey line is one sample travelling from noise (blue) to data (orange).

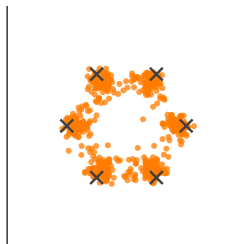
The denoising path obeys an ordinary differential equation:

$$\frac{d\mathbf{x}}{dt} = \frac{\mathbf{x} - D_{\theta}(\mathbf{x}; t)}{t}$$

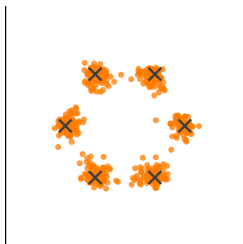
- ✱ $\mathbf{x}$: the image being formed
- ✱ t : noise level, run from $t = T$ down to $t = 0$
- ✱ D_{θ} : the denoiser from the last slide

Generating an image = solving this ODE numerically.

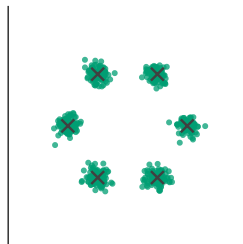
Fewer steps, wrong answer



NFE = 4



NFE = 10



NFE = 200 (reference)

How to read it: Crosses mark where the data actually lies. Dots are generated samples. The tighter the dots sit on the crosses, the better.

- ✱ The solver takes N steps, so it spends N NFE.
- ✱ Too few steps and the samples land in the wrong place.

Takeaway

The error we see is **truncation error** of the ODE solver. This is a numerical methods problem, not a machine learning one.

Can we cut the truncation error
without taking more steps?

- ✱ More steps would work, but steps are exactly what we cannot afford.
- ✱ So we need a better answer from the same network calls.

Richardson extrapolation

The classical idea

Suppose a method gives an estimate $V(h)$ with step size h , and we know how the error behaves:

$$V^* = V(h) + c h^p + O(h^q), \quad p < q$$

- ✱ V^* : the exact answer, unknown
- ✱ $V(h)$: what the method returns, known
- ✱ c : an unknown constant p : the order of the method

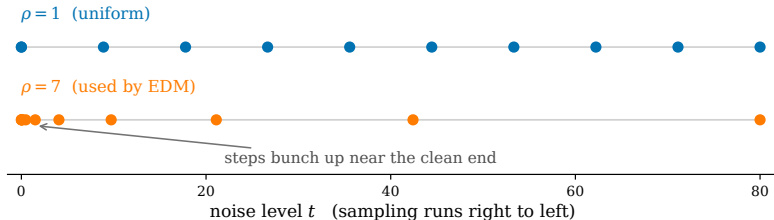
Run it **twice**, at h and at h/k . Two equations, two unknowns (V^* and c). Eliminate c :

$$\tilde{V} = \frac{k^p V(h/k) - V(h)}{k^p - 1}$$

Takeaway

Two wrong answers combine into a better one. The leading error term cancels. Stacking this repeatedly is **Romberg integration**.

The catch for diffusion models



How to read it: Each dot is one noise level the solver stops at. Top row: evenly spaced. Bottom row: what diffusion models actually use.

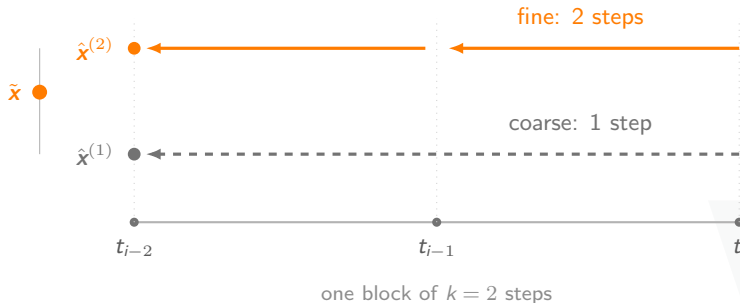
- ✱ Classical Richardson extrapolation assumes a **uniform** grid.
- ✱ Diffusion schedules are deliberately non-uniform: accuracy near the clean end matters more, so the steps crowd there.

Takeaway

The textbook coefficients do not apply. They need to be rederived for the actual step widths.

The base paper: RX-DPM

RX-DPM, in one picture



- ✱ Solve the **same** interval two ways: one coarse step, k fine steps.
- ✱ Both are wrong, but their errors are related. Combine them to cancel the leading term.

Choi, Kang & Han, *Enhanced Diffusion Sampling via Extrapolation with Multiple ODE Solutions*, ICLR 2025.

The grid-aware coefficients

Write the block width and the normalised sub-step widths as

$$h = t_i - t_{i-k}, \quad \lambda_j = \frac{t_{i-j+1} - t_{i-j}}{h}, \quad \sum_j \lambda_j = 1.$$

The two estimates carry errors that differ only through the **moment**

$$S_p = \sum_{j=1}^k \lambda_j^p:$$

$$\hat{\mathbf{x}}^{(1)} = \mathbf{x}^* + c h^p + O(h^q) \quad (\text{one step})$$

$$\hat{\mathbf{x}}^{(k)} = \mathbf{x}^* + c S_p h^p + O(h^q) \quad (k \text{ steps})$$

Eliminating c gives the method:

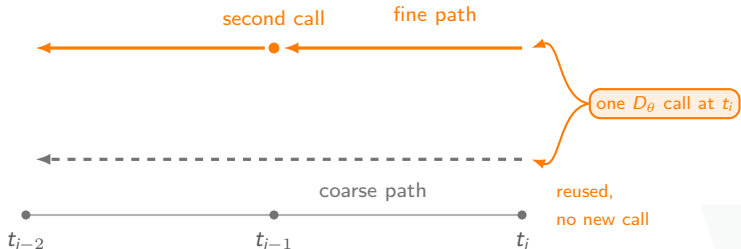
$$\tilde{\mathbf{x}} = \frac{\hat{\mathbf{x}}^{(k)} - S_p \hat{\mathbf{x}}^{(1)}}{1 - S_p}$$

$\mathbf{x}^*$: exact solution p : local order of the base solver ($p = 2$ for Euler) c : unknown constant

Takeaway

The coefficients depend on the schedule, through S_p . That is the paper's contribution.

Why it is free



- ✱ The coarse step needs a call **only at the start of the block**.
- ✱ The fine path already made it, so it is stored and reused.
- ✱ At the block start both paths sit at the same point, so the reuse is **exact**.

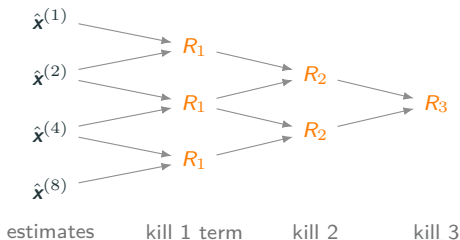
Takeaway

Better accuracy for **zero** extra NFE. This is why RX-DPM is attractive.

Our extension

Why stop at two?

Romberg integration does not stop at one extrapolation. It builds a tableau, each column cancelling one more error term.



- ✱ RX-DPM uses only the first column.
- ✱ Our extension: build the whole tableau, on the non-uniform grid.

The L -level system

Take L nested subdivisions $\Lambda_1, \dots, \Lambda_L$ of the same block (1, 2, 4, ... sub-steps). Extending the error model to further terms:

$$\hat{\mathbf{x}}^{(k_n)} = \mathbf{x}^* + \sum_{m=1}^{L-1} \mathbf{b}_m S_{p+m-1}(\Lambda_n), \quad n = 1, \dots, L$$

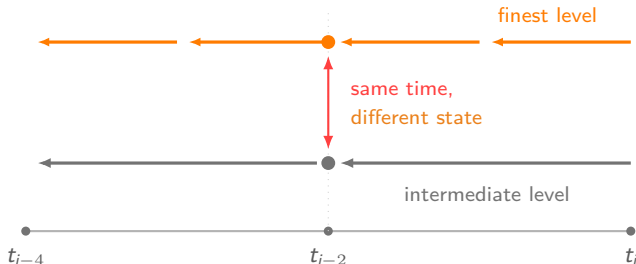
Λ_n : the n -th subdivision ■ k_n : how many sub-steps it has ■ $S_r(\Lambda) = \sum_{\lambda \in \Lambda} \lambda^r$ ■
 $\mathbf{b}_m$: unknown error coefficients

- ✱ That is L equations in L unknowns: a linear system M .
- ✱ The weights are the first row of M^{-1} , i.e. $M^T \mathbf{w} = \mathbf{e}_1$.
- ✱ We solve it with LU factorisation with partial pivoting.
- ✱ At $L = 2$ it reduces to the published formula exactly.

Takeaway

On a uniform grid M is a Vandermonde matrix and this is Romberg. On the real schedule the moments must be formed explicitly.

But the reuse trick stops working



- ✱ By t_{i-2} the two paths have **separated**.
- ✱ Reusing the stored prediction is now an **approximation**, not an identity.
- ✱ Recomputing honestly costs extra NFE, losing the whole selling point.

Takeaway

We implement both, and let the measurements decide.

Experiments

What we run on

An analytic problem

- ✦ Data is a Gaussian mixture, so D_θ is known in closed form.
- ✦ Same ODE, same sampler code, no network.
- ✦ The exact solution is available, so we can measure true error, not a proxy.

CIFAR-10 with EDM

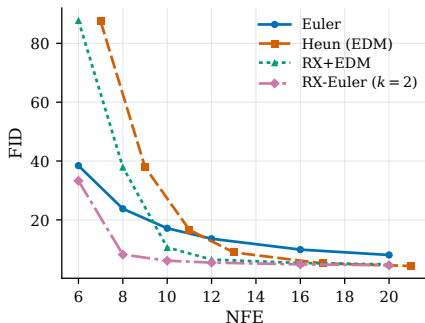
- ✦ Pretrained model; only the sampler changes.
- ✦ Quality measured by FID (lower is better).
- ✦ 10,000 images per FID, same seeds for every method.
- ✦ 3 seed blocks for error bars.

Correctness first

Our sampler matches the authors' published code to $< 10^{-12}$ across eight step/block combinations, at identical NFE.

Results

The reproduction works



How to read it: Left is cheaper, down is better.

We want curves low and to the left.

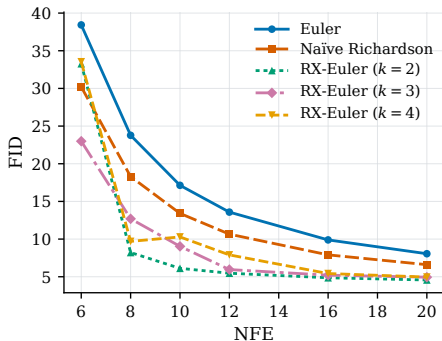
Takeaway

RX-DPM reproduces. The ordering matches the paper.

Method	NFE	FID
Euler	10	17.48
Heun (EDM)	11	16.45
RX-Euler	10	6.15
RX+EDM	10	10.72

- ✱ 2.8× better than Euler at the same cost.
- ✱ Beats Heun, which gets an extra evaluation.

Grid-aware coefficients matter



How to read it: Compare the classical curve against the grid-aware ones.

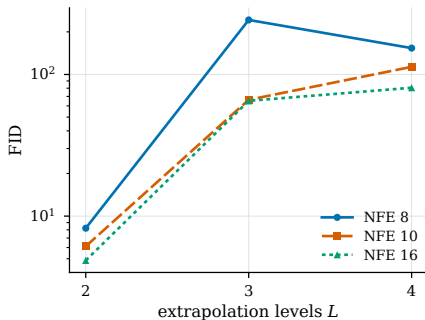
- ✦ At NFE = 10: classical 13.41 vs grid-aware **6.12**.
- ✦ A factor of 2.2, from the coefficients alone.
- ✦ $k=2$ is the best block length, as the paper reports.

At the very smallest budget the blocks are so wide that the error model no longer holds, and the classical coefficients edge ahead.

Takeaway

Adapting the coefficients to the actual step widths is what makes the method work.

Our extension: more levels is worse



How to read it: Vertical axis is logarithmic. Down is better.

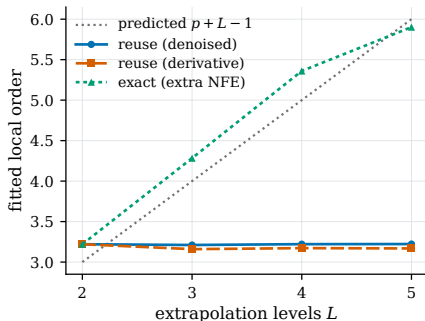
NFE	$L = 2$	$L = 3$	$L = 4$
8	8.21	242.8	153.5
10	6.12	66.3	113.2
16	4.86	65.2	80.6

- ✦ Three levels is an **order of magnitude worse**, not better.
- ✦ Same on the analytic problem, so it is not a quirk of FID.

Takeaway

The obvious generalisation fails. The rest of the talk is **why**.

The cause: the reuse ceiling



How to read it: Higher is better. Dotted line is what theory predicts.

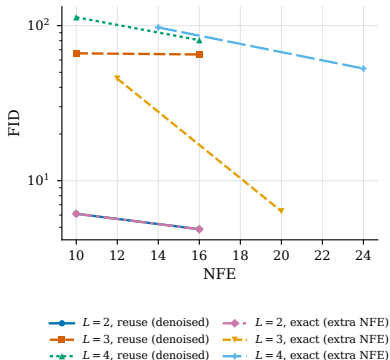
L	reuse	exact	predicted
2	3.22	3.22	3
3	3.21	4.28	4
4	3.22	5.36	5
5	3.22	5.90	6

- ✱ At $L = 2$ the two are **identical**: nothing is approximated.
- ✱ From $L = 3$, reuse is **stuck at 3.2** forever.
- ✱ Exact recomputation tracks theory.

Takeaway

The maths is right. The **reuse** is the ceiling, and it is exactly what made the method free.

And paying for it does not help



- ✦ Exact $L = 3$ reaches FID 6.35 at NFE = 20.
- ✦ Plain $L = 2$ reaches 4.57 at the same budget.
- ✦ The extra evaluations buy less than simply running two levels.

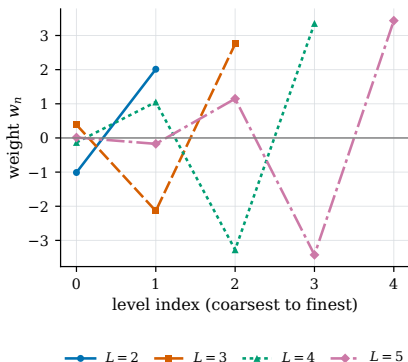
How to read it: Exact recomputation shifts a curve

right: it spends more NFE.

Takeaway

The free lunch of RX-DPM belongs to two levels and does not generalise.

Why the system also gets fragile



How to read it: Weights swing between positive and negative, and grow with L .

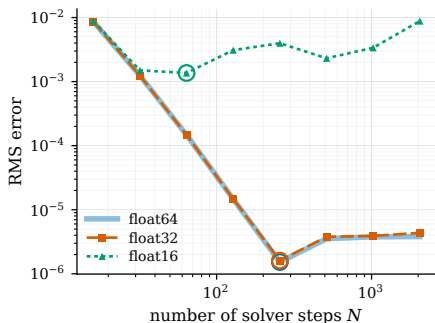
Takeaway

Adding levels means subtracting nearly equal large numbers. Classic cancellation.

L	$\kappa_{\infty}(M)$	$\sum_n w_n $
2	8.0	3.00
4	5.8×10^2	6.50
7	6.1×10^5	8.99

- ✦ κ_{∞} : how much the solve amplifies input error.
- ✦ Roughly $10\times$ worse per level.
- ✦ Pivot growth stays at 1.0, so this is the **problem**, not our solver.

But round-off is not the bottleneck



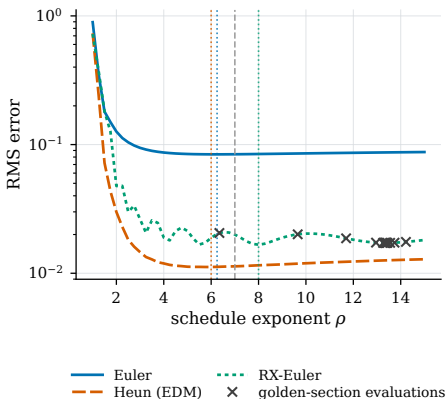
How to read it: Right is more work. Circles mark each curve's best point.

Takeaway

We expected precision to be the limit. It is not: at useful step counts, truncation still dominates.

- * Error falls, bottoms out, then **turns back up**: the classic trade-off.
- * But the turn happens near $N = 256$.
- * Real samplers use $N = 10$ to 20.
- * Half precision instead sets an error **floor**.

A small bonus: the schedule exponent



- ✱ ρ controls how steps bunch towards the clean end.
- ✱ EDM picked $\rho = 7$ for Euler and Heun.
- ✱ We search with **golden-section search**.
- ✱ Best for RX: $\rho \approx 6.2$, FID 5.95 vs 6.12.

Not cleanly unimodal, so we claim a good *region*, not a minimiser.

How to read it: Lower is better. The dashed grey line is the value EDM uses.

Takeaway

The schedule and the coefficients are better tuned together than separately.

Takeaways

What we found

1. **RX-DPM reproduces.** $2.8\times$ lower FID than Euler at equal cost, and the grid-aware coefficients are what make it work.
2. **Our extension fails, and we know exactly why.** The reuse that makes the method free is exact **only** at two levels. Past that it caps the order at 3.2.
3. **The free lunch does not scale.** Paying the extra evaluations restores the order but costs more than it returns.
4. **Round-off is not the limit.** The wall sits at $N = 256$; samplers live at $N = 10$ to 20.

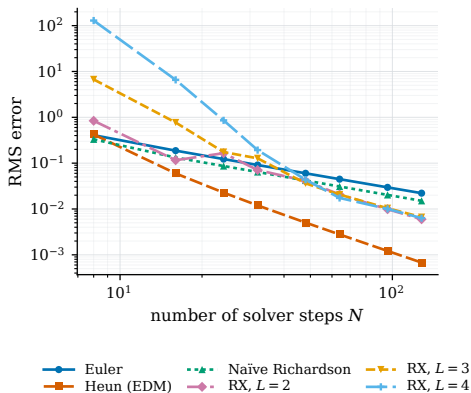
The shape of the result

A negative result with a measured cause is still a result: we located the boundary of a published method and said precisely where it lies.

Thank you

Questions?

Backup: convergence orders on the analytic problem

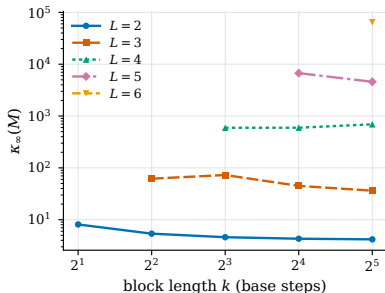
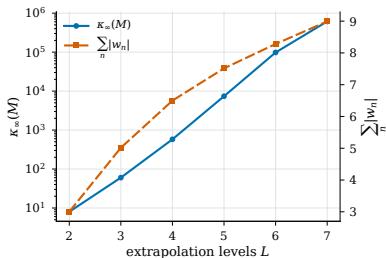


Method	fitted	predicted
Euler	1.02	1
Heun	2.05	2
Classical R.	1.04	2
RX ($L = 2$)	1.94	2

Euler and Heun hit their textbook

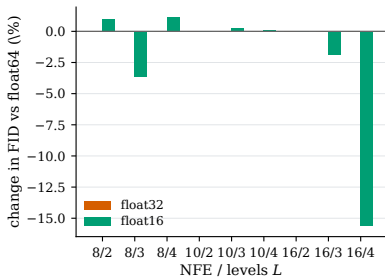
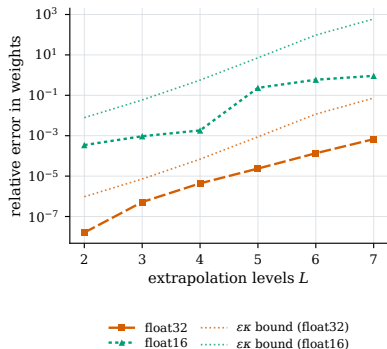
orders, which validates the
harness.

Backup: conditioning in detail



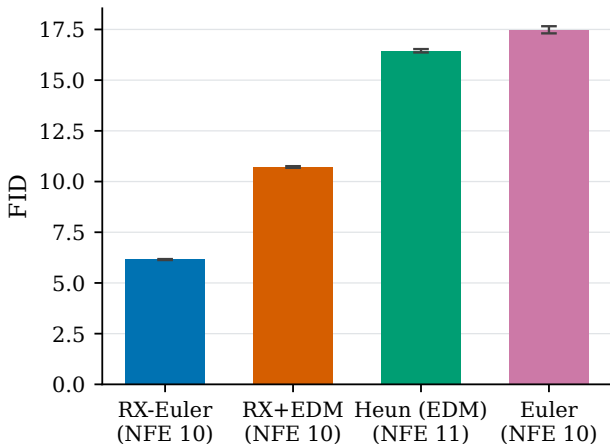
Left: conditioning against level count. Right: against block length, showing the growth is driven by **levels**, not by the spread of step widths.

Backup: precision studies



Left: error in the computed weights, with the ε_K bounds dotted. Right: the effect on FID, which is negligible at $L = 2$.

Backup: seed blocks and error bars



Spread over three disjoint seed blocks, shown as the standard error of the mean. Every gap discussed in the talk is far larger than the sampling variability.